

UNIVERSIDAD PRIVADA ANTENOR ORREGO - UPAO

FACULTAD DE INGENIERIA

SILABO DEL CURSO: ALGORITMIA Y PROGRAMACION

Carrera: Ingenieria de Sistemas e Inteligencia Artificial | Ciclo: I | Creditos: 4

1. INFORMACION GENERAL

Codigo del curso: CS101 | Horas teoricas: 2 | Horas practicas: 4 | Semestre: 2026-10

Prerrequisitos: Ninguno

2. SUMILLA

El curso de Algoritmia y Programacion es de naturaleza teorico-practica. Desarrolla en el estudiante las competencias para resolver problemas computacionales mediante el diseno de algoritmos y su implementacion en un lenguaje de programacion estructurado. Se abordan conceptos fundamentales como variables, tipos de datos, operadores, estructuras de control secuencial, condicional e iterativa, y la descomposicion de problemas complejos en subproblemas.

3. COMPETENCIAS DEL CURSO

Al finalizar el curso, el estudiante sera capaz de:

- Identificar y analizar problemas computacionales descomponiendolos en pasos logicos secuenciales.
- Diseñar algoritmos utilizando pseudocodigo y diagramas de flujo.
- Implementar soluciones usando estructuras de control: secuencial, condicional e iterativa.
- Depurar y corregir errores logicos y sintacticos en programas.

4. CONTENIDO TEMATICO

UNIDAD 1: FUNDAMENTOS DE ALGORITMIA (Semanas 1-3)

Tema 1.1: Introduccion a la algoritmia y pensamiento computacional

- Definicion de algoritmo: conjunto finito de pasos ordenados para resolver un problema.
- Caracteristicas de un algoritmo: preciso, definido y finito.
- Representacion de algoritmos: pseudocodigo y diagrama de flujo.

- Descomposicion de problemas: identificar entradas, procesos y salidas.

Tema 1.2: Variables y tipos de datos

- Variable: espacio en memoria que almacena un valor que puede cambiar.
- Tipos de datos basicos: entero (int), decimal (float), texto (string), logico (boolean).
- Declaracion e inicializacion de variables.
- Constantes: valores que no cambian durante la ejecucion del programa.

Tema 1.3: Operadores

- Operadores aritmeticos: suma (+), resta (-), multiplicacion (*), division (/), modulo (%).
- Operadores relacionales: igual (==), diferente (!=), mayor (>), menor (<), mayor o igual (>=), menor o igual (<=).
- Operadores logicos: AND (y), OR (o), NOT (no).
- Precedencia de operadores: orden en que se evaluan las operaciones.

UNIDAD 2: ESTRUCTURAS DE CONTROL (Semanas 4-8)

Tema 2.1: Estructura secuencial

- Definicion: las instrucciones se ejecutan una tras otra en orden.
- Entrada de datos: leer informacion del usuario.
- Procesamiento: realizar operaciones con los datos.
- Salida de datos: mostrar resultados al usuario.

Tema 2.2: Estructura condicional (If/Else)

- Condicional simple (if): ejecuta un bloque de codigo si la condicion es verdadera.
- Condicional doble (if/else): ejecuta un bloque si la condicion es verdadera, otro si es falsa.
- Condicional multiple (if/else if/else): evalua multiples condiciones.
- Condicionales anidadas: una estructura condicional dentro de otra.
- Ejemplo: determinar si un numero es positivo, negativo o cero.

Tema 2.3: Estructura iterativa - Bucle For

- Definicion: repite un bloque de codigo un numero determinado de veces.
- Componentes del bucle for: variable contadora, condicion de parada, incremento.
- Uso cuando se conoce de antemano el numero de repeticiones.
- Ejemplo: imprimir los numeros del 1 al 10.

- Bucles for anidados: un bucle dentro de otro para trabajar con matrices o tablas.

Tema 2.4: Estructura iterativa - Bucle While

- Definicion: repite un bloque de codigo mientras una condicion sea verdadera.
- Diferencia con el bucle for: se usa cuando no se conoce el numero exacto de repeticiones.
- Condicion de parada: debe modificarse dentro del bucle para evitar bucles infinitos.
- Ejemplo: pedir un numero al usuario hasta que ingrese un valor positivo.
- Bucle do-while: ejecuta el bloque al menos una vez antes de evaluar la condicion.

UNIDAD 3: DESCOMPOSICION Y SOLUCION DE PROBLEMAS (Semanas 9-12)

Tema 3.1: Metodologia para resolver problemas algoritmicos

- Paso 1 - Comprension del problema: identificar que se pide, que datos se tienen y que se debe obtener.
- Paso 2 - Analisis: descomponer el problema en subproblemas mas simples.
- Paso 3 - Diseno del algoritmo: escribir el pseudocodigo o diagrama de flujo.
- Paso 4 - Implementacion: codificar el algoritmo en el lenguaje de programacion.
- Paso 5 - Prueba y depuracion: verificar que el programa funciona correctamente con diferentes datos.

Tema 3.2: Depuracion de errores

- Error sintactico: incumplimiento de las reglas gramaticales del lenguaje.
- Error logico: el programa ejecuta pero no produce el resultado esperado.
- Error en tiempo de ejecucion: el programa se detiene inesperadamente durante su ejecucion.
- Estrategias de depuracion: usar trazas, imprimir valores intermedios, dividir el problema.

5. SISTEMA DE EVALUACION

Evaluacion continua (practicas de laboratorio): 40%

Examen parcial (semana 8): 30%

Examen final (semana 16): 30%

Nota minima aprobatoria: 11 sobre 20

6. BIBLIOGRAFIA

- Joyanes Aguilar, L. (2008). Fundamentos de programacion: algoritmos, estructura de datos y objetos. McGraw-Hill.

- Cormen, T., Leiserson, C., Rivest, R., & Stein, C. (2009). Introduction to Algorithms. MIT Press.
- Wing, J. M. (2006). Computational thinking. Communications of the ACM, 49(3), 33-35.